Ain Shams University
Faculty of Engineering
CSE354: Distributed Computing – 2nd Semester 2025/2026

This project is a group project, each group will be 2 to 5 students. Each group will deliver a written report in addition to the developed code by the end of the project that shows the detailed analysis, design, testing … etc. of the project and the description of the used techniques, problem description, detailed solution, limitations, sample output, references, and any additional information related to the project. Online Demo (YouTube) Video, presentation, and discussion will be conducted by the end of the project.

Introduction

This coursework is itemized into several parts to get the 35 marks associated to it. You must use the templates provided by the instructor to prepare your work.
All assignments and projects will be handed-in electronically, while quizzes and exams are written .

Learning Outcome to be assessed

1. Design a distributed computing model to solve a complex problem
2. Design and Implement a distributed computing model
3. Configure a working environment for distributed computing
4 Work and communicate effectively in a team

Marking Criteria 89% and above:

Your work must be of outstanding quality and fully meet the requirements of the coursework specification and learning outcomes stated. You must show independent thinking and apply this to your work showing originality and consideration of key issues. There must be evidence of wider reading on the subject. In addition, your proposed solution should:

- illustrate a professional ability of drafting construction details,
- express a deep understanding of the in-hand problem definition,
- and applying, masterly, the learned knowledge in the proposed solution.

76% - 89%:

Your work must be of good quality and meet the requirements of the coursework specification and learning outcomes stated. You must demonstrate some originality in your work and show this by applying new learning to the key issues of the coursework. There must be evidence of wider reading on the subject. In addition, your proposed solution should:

- illustrate a Good ability of drafting construction details,
- express a very Good understanding of the in-hand problem definition,
- and applying most of the learned knowledge, correctly, in the proposed solution.

67% - 76%:

Your work must be comprehensive and meet all of the requirements stated by the coursework specification

and learning outcomes. You must show a good understanding of the key concepts and be able to apply them to solve the problem set by the coursework. There must be enough depth to your work to provide evidence of wider reading. In addition, your proposed solution should:

- illustrate a moderate ability of drafting construction details,
- express a good understanding of the in-hand problem definition,
- and applying most of the learned knowledge, correctly, in the proposed solution.

60% - 67%:

Your work must be of a standard that meets the requirements stated by the coursework specification and learning outcomes. You must show a reasonable level of understanding of the key concepts and principles and you must have applied this knowledge to the coursework problem. There should be some evidence of wider reading. In addition, your proposed solution should:

- illustrate a fair ability of drafting construction details,
- express a fair understanding of the in-hand problem definition,
- and applying some of the learned knowledge, correctly, in the proposed solution.

Below 60%:

Your work is of poor quality and does not meet the requirements stated by the coursework specification and learning outcomes. There is a lack of understanding of key concepts and knowledge and no evidence of wider reading. In addition, your proposed solution would be:

- Illustrate an inability of drafting construction details,
- Failed to define the parameters, limitations, and offerings of the in-hand problem,
- Failed to apply correctly the learned knowledge for proposing a valid solution.

Academic Misconduct

The University defines Academic Misconduct as 'any case of deliberate, premeditated cheating, collusion, plagiarism or falsification of information, in an attempt to deceive and gain an unfair advantage in assessment'. This includes attempting to gain marks as part of a team without making a contribution. The department takes Academic Misconduct very seriously and any suspected cases will be investigated through the University's standard policy. If you are found guilty, you may be expelled from the University with no award.

It is your responsibility to ensure that you understand what constitutes Academic Misconduct and to ensure that you do not break the rules. If you are unclear about what is required, please ask.

Title: Efficient Load Balancing and GPU Cluster Task Distribution for Handling 1000+ Concurrent LLM Requests

Description: This project aims to design and implement a distributed system capable of handling 1000+ concurrent user requestsinvolving Large Language Model (LLM) inference and Retrieval-Augmented Generation (RAG).
The system focuses on efficient load balancing and task distribution across GPU clusters to ensure high performance, low latency, and optimal resource utilization.
The implementation will simulate real-world AI workloads where requests require heavy computation and must be distributed dynamically across multiple processing nodes. A key focus will be on scalability, performance optimization, and fault tolerance in distributed environments.

Features and Specifications:

● **Load Balancing Mechanism:**
The system should distribute incoming user requests efficiently across multiple compute nodes using strategies such as:

- Round Robin
- Least Connections
- Load-aware routing

● **GPU Cluster Task Distribution:**

- Assign LLM inference tasks across multiple GPU nodes
- Optimize GPU utilization and minimize idle time
- Support parallel processing of requests

● **LLM Inference Handling:**

- Process user queries using LLM models
- Handle high concurrency efficiently

● **RAG Integration:**

- Retrieve relevant data from a knowledge base
- Enhance LLM responses with contextual information

● **Scalability:**

- System should handle increasing number of users (up to 1000+)
- Efficient scaling of compute resources

● **Fault Tolerance:**

- Detect node failures
- Reassign tasks automatically
- Maintain system availability

User Stories:

- As a user, I want to send multiple AI requests simultaneously and receive fast responses.
- As a user, I want the system to remain responsive even under heavy load.
- As a user, I want the system to continue functioning even if some nodes fail.
- As a user, I want consistent and accurate responses from the LLM system.
- As a system administrator, I want to monitor performance and resource utilization.

System Architecture:

- **Client Layer**
  - Simulates 1000 concurrent users
  - Sends requests to the system
- **Load Balancer**
  - Receives incoming requests
  - Distributes them across compute nodes
  - Implements scheduling strategies
- **Master Node (Controller)**
  - Manages task scheduling
  - Assigns requests to GPU nodes
  - Monitors system performance
- **GPU Worker Nodes**
  - Execute LLM inference tasks
  - Process requests in parallel
  - Return results to master node
- **RAG Module**
  - Retrieves relevant data
  - Enhances LLM responses

Fault Tolerance Mechanisms to Implement:

● **Worker Node Failure:**

- Detect failed GPU nodes
- Reassign tasks to active nodes

● **Task Reassignment:**

- Ensure no request is lost
- Maintain processing continuity

● **Load Balancer Resilience:**

- Continue distributing requests even under partial failure

Technology Suggestions:

● Programming Language: Python

● LLM Frameworks:

- PyTorch
- TensorFlow

● Load Balancing Tools:

- NGINX
- HAProxy

● GPU Computing:

- CUDA
- NVIDIA GPU libraries

● Data Handling:

- Vector databases (for RAG)

Implementation Steps:

1. System Design and Architecture Definition
2. Load Balancer Implementation
3. GPU Task Distribution Logic
4. LLM Inference Integration
5. RAG Pipeline Implementation
6. Performance Optimization
7. Fault Tolerance Implementation
8. Testing and Evaluation
9. Documentation

Testing and Evaluation (Focus on Fault Tolerance):

● **Load Testing:**

- Simulate 100 → 1000 concurrent users

● **Performance Metrics:**

- Latency
- Throughput
- GPU utilization

● **Failure Simulation:**

- Shut down nodes during execution
- Verify system recovery

Project Phases

Phase 1: Architecture Design

- Define system components
- Design data flow
- Select technologies

Phase 2: Core Implementation

- Implement load balancing
- Implement GPU task distribution
- Basic LLM inference

Phase 3: Enhancement & Fault Tolerance

- Add RAG integration
- Implement fault tolerance
- Improve performance

Phase 4: Testing & Finalization

- Full system testing
- Performance evaluation
- Documentation and presentation

Code Skeleton

- Request/Response

```python
# common/models.py

from dataclasses import dataclass

@dataclass
class Request:
    id: int
    query: str

@dataclass
class Response:
    id: int
    result: str
    latency: float
```

- Load Balancer (Round Robin)

```python
# lb/load_balancer.py

class LoadBalancer:
    def __init__(self, workers):
        self.workers = workers
        self.index = 0

    def get_next_worker(self):
        worker = self.workers[self.index]
        self.index = (self.index + 1) % len(self.workers)
        return worker

    def dispatch(self, request):
        worker = self.get_next_worker()
        return worker.process(request)
```

- Master Scheduler

```python
# master/scheduler.py

class Scheduler:
    def __init__(self, load_balancer):
        self.lb = load_balancer

    def handle_request(self, request):
        print(f"[Scheduler] Dispatching request {request.id}")
        response = self.lb.dispatch(request)
        return response
```

- GPU Worker Node

```python
# workers/gpu_worker.py

import time
from llm.inference import run_llm
from rag.retriever import retrieve_context

class GPUWorker:
    def __init__(self, id):
        self.id = id

    def process(self, request):
        start = time.time()

        print(f"[Worker {self.id}] Processing request {request.id}")

        # RAG Step
        context = retrieve_context(request.query)

        # LLM Step
        result = run_llm(request.query, context)

        latency = time.time() - start

        return {
            "id": request.id,
            "result": result,
            "latency": latency
        }
```

- RAG Retriever (Stub)

```
# rag/retriever.py

def retrieve_context(query):
    # Simulate vector DB retrieval
    return f"Relevant context for: {query}"
```

- LLM Inference (Stub)

```
# llm/inference.py

import time

def run_llm(query, context):
    # Simulate GPU inference delay
    time.sleep(0.2)

    return f"LLM Answer to '{query}' using [{context}]"
```

- Client Load Generator (1000 Users)

```
# client/load_generator.py

import threading
from common.models import Request

def simulate_user(scheduler, user_id):
    request = Request(id=user_id, query=f"Query {user_id}")
    response = scheduler.handle_request(request)

    print(f"[Client] Response {response['id']} | Latency: {response['latency']:.3f}s")


def run_load_test(scheduler, num_users=1000):
    threads = []

    for i in range(num_users):
        t = threading.Thread(target=simulate_user, args=(scheduler, i))
        threads.append(t)
        t.start()

    for t in threads:
        t.join()
```

- Main Entry Point

```python
# main.py

from workers.gpu_worker import GPUWorker
from lb.load_balancer import LoadBalancer
from master.scheduler import Scheduler
from client.load_generator import run_load_test

def main():
    # Create GPU workers
    workers = [GPUWorker(i) for i in range(4)]  # simulate 4 GPUs

    # Load Balancer
    lb = LoadBalancer(workers)

    # Scheduler
    scheduler = Scheduler(lb)

    # Run simulation
    run_load_test(scheduler, num_users=1000)

if __name__ == "__main__":
    main()
```

How to run the above skeleton:

- Install Python 3.9+ and verify with python --version
- Create the project folders (client, master, workers, lb, rag, llm, common)
- Create all required .py files inside each folder
- Copy the provided code into the corresponding files
- Open terminal and navigate to the project root folder
- (Optional) Set PYTHONPATH=. if imports fail
- Run the project using python main.py
- Observe logs showing scheduler, load balancer, and workers processing requests
- Adjust number of users in run_load_test() for load testing
- Adjust number of workers in main.py to simulate more/less GPUs